

Tarefa 10: Mecanismos de Sincronização com OpenMP: Estimativa de π via Monte Carlo: critical, atomic e reduction

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Objetivo

Comparar cinco mecanismos de sincronização do OpenMP no contexto do estimador Monte Carlo de π , todos utilizando `rand_r()` com seed privada por thread para eliminar contenção no gerador de números aleatórios. As versões avaliadas são: contador compartilhado protegido por `critical`, contador compartilhado protegido por `atomic`, contador privado por thread acumulado com `critical`, contador privado por thread acumulado com `atomic`, e a cláusula `reduction`. Ao final, propõe-se um roteiro de decisão para escolha do mecanismo adequado conforme o padrão de acesso.

2. Ambiente de Execução

Item	Valor
CPU	AMD Ryzen 5 5600G (4 núcleos)
Threads	4
N (pontos)	100.000.000
Compilador	GCC -O2 -fopenmp
Sistema Operacional	Linux (WSL2)

3. Implementação

Todas as versões utilizam `rand_r()` com seed privada por thread para eliminar contenção no gerador de números aleatórios. A única variável que muda entre as versões é o mecanismo de sincronização aplicado ao contador de acertos (pontos dentro do círculo unitário).

3.1 V1 — Contador compartilhado + critical

Cada acerto adquire o mutex global da seção crítica. Como $\pi/4 \approx 0,785$, ocorrem aproximadamente 78 milhões de aquisições de mutex, serializando cerca de 78% das iterações.

```

long hits = 0;
#pragma omp parallel
{
    unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
    #pragma omp for
    for (long i = 0; i < N; i++) {
        double x = rand_r(&seed) / (double)RAND_MAX;
        double y = rand_r(&seed) / (double)RAND_MAX;
        if (x*x + y*y <= 1.0) {
            #pragma omp critical /* ~78M aquisições de mutex */
            hits++;
        }
    }
}

```

3.2 V2 — Contador compartilhado + atomic

Idêntica à V1, mas a proteção do incremento usa atomic em vez de critical: uma instrução atômica de hardware (LOCK XADD em x86), sem mutex e sem syscall.

```

if (x*x + y*y <= 1.0) {
    #pragma omp atomic /* ~78M operações atômicas de hardware */
    hits++;
}

```

3.3 V3 — Contador privado + critical

Cada thread acumula em uma variável local, sem qualquer sincronização durante o loop. A seção crítica é executada apenas uma vez por thread (4 vezes no total), ao somar o resultado local no contador global.

```

long local = 0;
for (long i = 0; i < N/4; i++) { /* ... */ if (/* dentro do círculo */) local++; }
#pragma omp critical /* 4 aquisições no total */
hits += local;

```

3.4 V4 — Contador privado + atomic

Mesma estratégia da V3, porém a soma final usa atomic em vez de critical. Como ocorre apenas 4 vezes, a diferença frente à V3 é mínima.

```

long local = 0;
for (long i = 0; i < N/4; i++) { /* ... */ if (/* dentro do círculo */) local++; }
#pragma omp atomic /* 4 operações atômicas no total */
hits += local;

```

3.5 V5 — reduction

O OpenMP gera internamente contadores privados por thread e realiza a redução final de forma automática — zero sincronização explícita no código-fonte.

```

long hits = 0;
#pragma omp parallel for reduction(+:hits)
for (long i = 0; i < N; i++) {
    unsigned seed = SEED ^ ((unsigned)omp_get_thread_num()
                           * 2654435761u + (unsigned)i);
    double x = rand_r(&seed) / (double)RAND_MAX;

```

```
double y = rand_r(&seed) / (double)RAND_MAX;
if (x*x + y*y <= 1.0) hits++;
}
```

4. Resultados

Versão	π estimado	Tempo (s)	Sync/iter
V1 shared + critical	3,14159932	6,45	~78M mutex
V2 shared + atomic	3,14159932	1,10	~78M atomic
V3 private + critical	3,14159932	0,27	4 mutex
V4 private + atomic	3,14159932	0,26	4 atomic
V5 reduction	3,14159396	0,21	0 (interno)

5. Análise

5.1 V1 vs V2 — critical vs atomic no caminho quente

Ambas as versões sincronizam a cada acerto (cerca de 78 milhões de vezes). A diferença de desempenho é drástica: 6,45 s contra 1,10 s, uma razão de aproximadamente 5,9×. A diretiva `critical` utiliza um mutex do sistema operacional — `lock`, incremento e `unlock` — em que cada aquisição pode envolver uma chamada de sistema, uma barreira de memória completa e contenção entre as quatro threads. Já a diretiva `atomic` emite uma instrução de hardware do tipo `LOCK XADD`: é lock-free e executada inteiramente em nível de hardware, sem entrar no kernel, embora ainda invalide a linha de cache da variável `hits` em todos os núcleos a cada operação.

Conclui-se que `atomic` é muito mais rápido que `critical` para operações simples de alta frequência, mas ambas as abordagens são inadequadas quando a sincronização ocorre em quase toda iteração do loop.

5.2 V3 vs V4 — critical vs atomic com sincronização rara

Com contadores privados, a sincronização cai para apenas 4 operações — uma por thread, executada ao final do trabalho local. Nesse cenário, V3 (0,27 s) e V4 (0,26 s) apresentam desempenho praticamente equivalente: o custo de 4 aquisições de mutex versus 4 operações atômicas é desprezível na escala total da execução. O gargalo de desempenho passa a ser o próprio cálculo matemático (geração de números aleatórios e verificação do círculo), e não mais a sincronização.

5.3 V3/V4 vs V1/V2 — o impacto da frequência de sincronização

A diferença entre V2 e V4 (1,10 s contra 0,26 s, razão de 4,2×) ilustra a lição central deste experimento: a frequência de sincronização importa mais do que o tipo de mecanismo escolhido. Substituir `atomic` por `critical` em V1 custou um fator de 5,9×; substituir sincronização frequente

por sincronização rara custou um fator de 4,2× — mantendo o mesmo mecanismo de sincronização. Isso evidencia que reduzir a frequência das operações sincronizadas é uma otimização tipicamente mais impactante do que apenas trocar o tipo de primitiva utilizada.

5.4 V5 — reduction como abstração ótima

A versão V5 (0,21 s) é simultaneamente a mais rápida e a mais simples de todas. O compilador e o runtime do OpenMP implementam, por trás da cláusula reduction, exatamente o mesmo padrão empregado manualmente em V3 e V4 — acumulador local por thread seguido de redução final — porém sem exigir nenhum código de sincronização explícito no código-fonte. Além de mais rápida, essa abordagem é mais legível, menos propensa a erros e mais portátil entre diferentes implementações de OpenMP.

6. Roteiro de Decisão

Com base nos resultados obtidos, propõe-se o seguinte fluxo de decisão para escolha do mecanismo de sincronização em código OpenMP:

```
Precisa agregar um valor em um loop paralelo?
-> USE reduction: mais rápido, mais legível, primeira escolha sempre.

A expressão a proteger é simples (++ , += , *= , &= , |=)?
-> Frequência alta (todo ou quase todo o loop): use atomic.
-> Frequência baixa (poucas vezes por thread): critical ou atomic, indiferente.
-> Melhor ainda: acumule localmente e use atomic/reduction apenas no final.

A seção crítica é complexa (múltiplas instruções, chamadas de função)?
-> Recurso único compartilhado: #pragma omp critical (sem nome).
-> Recursos distintos acessáveis em paralelo: #pragma omp critical(nome).
    Cada nome é um mutex independente; inserir na lista A não bloqueia a lista B.
-> Número de recursos definido em tempo de execução (M dinâmico):
    critical(nome) exige identificador fixo em tempo de compilação;
    use omp_lock_t, alocável dinamicamente e indexável por variável.
    Também necessário para: trylock, seções longas, hierarquia de locks.
```

Mecanismo	Granularidade	Frequência ideal	Complexidade da seção
reduction	loop inteiro	implícita	só agregação
atomic	instrução	alta	1 operação simples
critical (sem nome)	bloco	baixa	qualquer
critical(nome)	bloco	baixa	qualquer (M fixo)
omp_lock_t	runtime	qualquer	qualquer (M dinâmico)

7. Conclusão

O experimento confirma que a escolha do mecanismo de sincronização em OpenMP deve considerar, antes de tudo, a frequência com que a sincronização ocorre, e não apenas o tipo de primitiva. Quando a sincronização acontece em quase toda iteração (V1 e V2), o custo domina o tempo total de execução, e mesmo a alternativa mais rápida (atomic) ainda é cerca de 5× mais lenta que as versões com acumulação local. Quando a sincronização é reduzida a poucas operações por thread (V3 e V4), a diferença entre critical e atomic torna-se irrelevante, pois o gargalo passa a ser o próprio trabalho computacional. Por fim, a cláusula reduction se mostrou superior em todos os aspectos avaliados: mais rápida, mais legível e mais segura, devendo ser a primeira escolha sempre que o padrão de acumulação permitir seu uso. Para os demais casos, em que a lógica protegida é mais complexa do que uma simples soma, a escolha recai sobre critical, critical(nome) ou omp_lock_t, conforme o número e a natureza dos recursos compartilhados.

7. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <time.h>
6  #include <omp.h>
7
8  #define N      100000000L
9  #define NTHREADS 4
10 #define SEED    42u
11 #define PI_REF  3.14159265358979323846
12 #define BILLION 100000000.0
13
14 static double elapsed(struct timespec a, struct timespec b) {
15     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
16 }
17
18 /* — V1: contador compartilhado + #pragma omp critical ————— */
19 static double v1_shared_critical(void) {
20     long hits = 0;
21     omp_set_num_threads(NTHREADS);
22     #pragma omp parallel
23     {
24         unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
25         #pragma omp for
26         for (long i = 0; i < N; i++) {
27             double x = rand_r(&seed) / (double)RAND_MAX;
28             double y = rand_r(&seed) / (double)RAND_MAX;
29             if (x*x + y*y <= 1.0) {
30                 #pragma omp critical
31                 hits++;
32             }
33         }
34     }
35     return 4.0 * hits / N;
36 }
37
38 /* — V2: contador compartilhado + #pragma omp atomic ————— */
39 static double v2_shared_atomic(void) {
40     long hits = 0;
41     omp_set_num_threads(NTHREADS);
42     #pragma omp parallel
43     {
44         unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
45         #pragma omp for
46         for (long i = 0; i < N; i++) {
47             double x = rand_r(&seed) / (double)RAND_MAX;
48             double y = rand_r(&seed) / (double)RAND_MAX;
49             if (x*x + y*y <= 1.0) {
50                 #pragma omp atomic
51                 hits++;
52             }
53         }
54     }
55     return 4.0 * hits / N;
56 }
57
```

```

57
58 /* — V3: contador privado + #pragma omp critical ————— */
59 static double v3_private_critical(void) {
60     long hits = 0;
61     omp_set_num_threads(NTHREADS);
62     #pragma omp parallel
63     {
64         unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
65         long local = 0;
66         for (long i = 0; i < N / NTHREADS; i++) {
67             double x = rand_r(&seed) / (double)RAND_MAX;
68             double y = rand_r(&seed) / (double)RAND_MAX;
69             if (x*x + y*y <= 1.0) local++;
70         }
71         #pragma omp critical
72         hits += local;
73     }
74     return 4.0 * hits / N;
75 }
76
77 /* — V4: contador privado + #pragma omp atomic ————— */
78 static double v4_private_atomic(void) {
79     long hits = 0;
80     omp_set_num_threads(NTHREADS);
81     #pragma omp parallel
82     {
83         unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
84         long local = 0;
85         for (long i = 0; i < N / NTHREADS; i++) {
86             double x = rand_r(&seed) / (double)RAND_MAX;
87             double y = rand_r(&seed) / (double)RAND_MAX;
88             if (x*x + y*y <= 1.0) local++;
89         }
90         #pragma omp atomic
91         hits += local;
92     }
93     return 4.0 * hits / N;
94 }
95
96 /* — V5: reduction ————— */
97 static double v5_reduction(void) {
98     long hits = 0;
99     omp_set_num_threads(NTHREADS);
100     #pragma omp parallel for reduction(+:hits)
101     for (long i = 0; i < N; i++) {
102         unsigned seed = SEED ^ ((unsigned)omp_get_thread_num() * 2654435761u
103                                 + (unsigned)i);
104         double x = rand_r(&seed) / (double)RAND_MAX;
105         double y = rand_r(&seed) / (double)RAND_MAX;
106         if (x*x + y*y <= 1.0) hits++;
107     }
108     return 4.0 * hits / N;
109 }

```

```

110
111 int main(void) {
112     struct timespec ini, fim;
113
114     printf("Estimativa de  $\pi$  – Monte Carlo (N=%ldM, %d threads)\n\n",
115           N / 1000000L, NTHREADS);
116     printf("%-30s  %-14s  %-10s  %-10s\n",
117           "Versão", "π estimado", "Tempo(s)", "Sync/iter");
118     printf("%-30s  %-14s  %-10s  %-10s\n",
119           "-----", "-----",
120           "-----", "-----");
121
122     #define MEDIR(label, sync_desc, expr) \
123     do { \
124         clock_gettime(CLOCK_MONOTONIC, &ini); \
125         double pi_r = (expr); \
126         clock_gettime(CLOCK_MONOTONIC, &fim); \
127         printf("%-30s  %-14.8f  %-10.4f  %s\n", \
128               (label), pi_r, elapsed(ini, fim), (sync_desc)); \
129     } while (0);
130
131     MEDIR("V1 shared+critical", "~78M mutex", v1_shared_critical())
132     MEDIR("V2 shared+atomic", "~78M atomic", v2_shared_atomic())
133     MEDIR("V3 private+critical", "4 mutex", v3_private_critical())
134     MEDIR("V4 private+atomic", "4 atomic", v4_private_atomic())
135     MEDIR("V5 reduction", "0 (impl.)", v5_reduction())
136
137     printf("\nπ referência: %.8f\n", PI_REF);
138     return 0;
139 }
140

```